

A tool that reads 500 emails so its user doesn't have to

repo: github.com/chandrameenamohan/aubade (private) · verify everything with one command: `make check`

The problem

Avery Chen is a (fictional) startup CEO with ~500 emails over 30 days, a crowded calendar, meeting notes, and a task list. Every morning at 6:00 they have 90 seconds. They want one page that answers three questions: **What needs me today? What am I about to drop? What can I dispatch right now?** — and they want the page to admit when it isn't sure, instead of faking confidence.

The approach: a tool, not an application

We built `aubade` as a command-line tool — no app, no web page — for four reasons:

- **The engine is what's graded.** A CLI puts all the effort where the assignment points: reading, detecting, composing — not screens.
- **Agent Experience (AX).** The tool is designed so that AI agents are first-class users, not just humans. It detects *who is calling* — a person, Claude Code, Cursor, another agent — and answers accordingly: readable markdown for a human, structured JSON with hints for an agent. (This follows the design of the `ox / agentx` reference tools.) Any framework that can run a shell command — Claude Code, LangGraph, a cron job — can use `aubade` without any special integration.
- **Scriptable and schedulable.** A morning digest is a batch job. A CLI drops into cron or GitHub Actions unchanged.
- **Verifiable in pieces.** Every internal step is exposed as its own subcommand, so a grader can run and inspect each part separately.

Where the tool's methods came from

We didn't invent the feature list. Avery's profile (`profile.md`, supplied by the assignment) states, in their own words, what they're afraid of missing. **Each worry became one detector** — a small, deterministic function that scans the data for exactly that problem. Seven worries, seven detectors:

DETECTOR	AVERY'S OWN WORDS (FROM PROFILE.MD)	WHAT THE CODE LOOKS FOR
commitments	<i>"Promises I made in email and didn't put on a todo list. I'll send that by Friday" hiding in a Tuesday thread."</i>	Things Avery promised in sent mail with a deadline, and never delivered.
quiet-threads	<i>"If Marcus or another VC went quiet, I want to know after three business days."</i>	Important people who stopped replying — investors past 3 business days, customers whose reply pace suddenly slowed.
conflicts	<i>"Calendar collisions involving Wren's daycare, doctor appointments... Family first. Always."</i>	Double-booked meetings, meetings scheduled over protected deep-work hours, family events colliding with work.
contradictions	<i>"If two sources disagree (calendar says Friday, email says Monday), tell me — don't pick one and hide it."</i>	The same meeting described differently by email and calendar. Both versions are shown; the tool never guesses.
dispatchables	<i>"What can I dispatch right now? Replies under 3 sentences. Quick yes/no approvals."</i>	Emails answerable in under a minute — with a reply pre-drafted in Avery's voice, ready to send.
suppressions	<i>"Newsletters. Even the good ones. Marketing emails. Anything where the only action is FYI."</i>	What to keep OFF the page. Just as important as what goes on it.

staleness

"If the inbox data is older than 24 hours, say so."

Whether the data itself is fresh enough to trust — if not, the digest opens by saying so.

How a digest gets made — four steps

- 1 Load.** The four sources (emails, calendar, notes, tasks) arrive in four different file formats. A loader converts them all into one shared in-memory shape — the **Event model**. That's all "Event model" means: a common representation, so every detector can read every source the same way. It also means swapping fake data for real Gmail later changes one loader, nothing else.
- 2 Detect.** The seven detectors scan the corpus and emit **signals** — small findings ("Avery promised Marcus the cap table Tuesday; no follow-up exists"). Every signal carries a **citation**: the exact email id, calendar entry, or note it came from. Detectors are plain deterministic code — same data in, same signals out, every time. No AI is involved in establishing facts.
- 3 Compose.** Two modes turn signals into the one-page digest:
 - **With AI (default):** a model — the claude CLI today — is handed the signals and the detectors as tools. It decides what deserves the top slot, reads long threads before ranking them, and writes the page in Avery's voice.
 - **Without AI (`--no-llm`):** the same signals rendered through a fixed template. Plainer prose, identical facts, zero setup, zero API keys — this is the mode a grader can always run, and the mode our automated checks gate on.
- 4 Validate.** Before the page is accepted, every citation on it is checked against the signals from step 2. If the AI wrote a single line citing something no detector produced, the whole page is rejected and the template version is used instead. **The model chooses and phrases; it cannot invent.**

Using it

```
$ aubade-lab generate --seed 42 --today 2026-08-30 --out data/ # create the synthetic inbox (~500 emails)
$ aubade digest --no-llm # the zero-key digest — one page, every line cited
$ aubade digest # the AI-composed digest (uses claude CLI if installed)
$ aubade digest --customize brief.md # reshape it ("top 3 only, as bullets") — facts and honesty warnings survive
$ aubade tool commitments --json # run ONE detector directly; agents use these as their tool surface
$ aubade-lab eval # grade the digest against the answer key (next page)
```

How it integrates with Claude Code, SDKs, and other frameworks

Inside `aubade digest` the AI sits behind a small socket called the **Runner interface**: two capabilities, "answer one structured question" and "drive the tools to compose a page." Anything that can do those can be the digest's brain:

- **Today:** the claude CLI (the assignment blesses CLI runtimes; no API keys to manage). A codex-CLI runner participates in voting.
- **A registration away:** the Anthropic SDK, Claude Agent SDK, OpenAI SDK, and Gemini SDK all support tool-calling, so each becomes a Runner by writing one adapter — no engine changes.
- **Week two:** `aubade mcp serve` would expose the detectors over MCP, the standard tool protocol — after which LangGraph and every other MCP-aware framework can use them natively.
- **Consensus:** for the two hardest judgment calls (is this ambiguous thread urgent? what is THE one thing right now?) `aubade` asks *every* installed runner and takes a vote. If they disagree, the item goes to the "I'm not sure" section with the evidence shown — a disagreement between models is treated as information, not noise.

How we know it works: the exam, the student, and the answer key

Avery is fictional, so the assignment requires us to *create* the inbox ourselves. That's a trap in disguise — if we generate boring data, the tool has nothing to prove. We turned it into the evaluation method:

- 1 **Write the exam.** While generating the 500 emails, we deliberately planted 20 problems: 14 the digest *must* catch (a broken promise to the lead investor, an investor gone quiet, a calendar contradiction...) and 6 it *must ignore* (newsletters, marketing, FYI-only threads). The realistic filler around them is the exam paper; the planted problems are the questions.
- 2 **Keep the answer key honest.** The same code that plants each problem also writes it into an answer-key file (`traps.json`): what was planted, where, and what the digest should say about it. Because the exam and its key are written by the same function in the same moment, they can never drift apart.
- 3 **The student sits the exam blind.** The digest tool reads the inbox cold — it never sees the answer key. (They're even separate programs: the user-facing `aubade` binary contains no evaluation code at all; the key and graders live only in `aubade-lab`.)
- 4 **Grade automatically.** `aubade-lab eval` compares the digest to the key: every must-catch problem present? every must-ignore item absent? It exits with an error if anything is missed — which is what lets a CI pipeline enforce quality on every change.

20 / 20

planted problems handled correctly in the no-AI mode — enforced on every commit

pass³ 20/20

AI mode, run 3 separate times: all 20 correct in all 3 runs (and we report best-of-3 separately — never one blended number)

52 / 0

citations on the AI-composed page / citations that failed to trace back to a real detector finding

We also grade the graders

A test suite that can't fail proves nothing. So `eval --sabotage` switches each detector off in turn and checks the score **drops** — if it doesn't, an alarm says our graders are blind:

DETECTOR SWITCHED OFF	PROBLEMS MISSED
quiet-threads	4
conflicts	3
dispatchables	2
commitments · contradictions · suppressions · staleness	1 each

Exams we didn't write, other brains

`eval --adversarial` asks an AI to invent *new* planted problems we never thought of, injects them into a copy of the data, and re-grades — so the tool also passes exams it didn't study for.

And because the eval grades **the output page**, not the AI that made it, it works as a harness benchmark: point any runner — claude, codex, an SDK, your own — at the same corpus, and the same answer key scores it. Swap the brain, keep the exam.

What we chose not to build

ALTERNATIVE	WHY WE SAID NO
One big AI prompt	Feed all 500 emails to a model and ask for a digest. Unfixable flaw: no way to tell a good digest from a confident hallucination. Our detector/citation split exists precisely so quality is checkable.
No AI at all	Provable but judgment-free — rules can't decide that a 14-message thread needs reading before ranking. Kept as the <code>--no-llm</code> mode: the fallback, and the control we compare the AI mode against.

RAG / vector database	500 emails fit in memory. Search infrastructure would add failure modes to solve a scale problem this corpus doesn't have.
Omnigent (meta-harness)	An alpha framework for orchestrating many AI harnesses — bigger machinery than this needs. We kept its one good idea (several AIs answering the same question) as ~200 lines of our own voting code.
SDK integration now	Adds API-key handling this week for no new capability. The Runner socket makes it a week-two adapter, not a rewrite.
Web UI / Slack bot	The assignment says the engine is graded, not screens. CLI in, markdown out.

With a second week

- **Real data:** a Composio-backed loader for actual Gmail/Calendar — the Event model means only the loader changes.
- **More brains:** SDK runners (Anthropic, OpenAI, Gemini) and `aubade mcp serve` for LangGraph-style frameworks.
- **Harder exams:** mutation testing — shuffle dates and senders; the planted problems must still be caught.
- **Delivery:** the scheduled 05:45 PT run (GitHub Actions cron, designed in `DESIGN.md` , deliberately not implemented this week per the assignment) — with `--no-llm` as the fallback so an expired API token produces a plainer page, never a missing one.

repo: github.com/chandraneenamohan/aubade · every claim above is reproducible: `make check` (build + tests + the 20/20 exam) · deeper docs in-repo: `HLD.md` (architecture) · `DESIGN.md` (decisions & scheduling) · `VERIFICATION.md` (what the gate does and doesn't prove) · `docs/EVAL-PRINCIPLES.md` (eval doctrine) · committed dataset: `datasets/seed-42/`